МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«_____»

Кафедра _____

ЛАБОРАТОРНАЯ РАБОТА № 5

по дисциплине «Основы алгоритмизации и программирование»

Хеширование и экспериментальное исследование методов разрешения коллизий

Вариант 6

Направление: 38.03.05
«Бизнес-информатика»
Курс: 2
Выполнил: студент группы БИ 2-1
ФИО: Голдаева Валерия Валентиновна
Проверил: _____

2026

1. Цель работы и формулировка задания

Цель работы: изучить принципы хеширования как способа организации данных с временем доступа O(1) в среднем, реализовать хеш-таблицу и экспериментально исследовать влияние хеш-функции и коэффициента заполнения на её производительность.

Задание варианта 6:

Параметр	Значение
Тип ключей	Строки (артикулы ABC-12345)
Хеш-функция	П — полиномиальная для строк
Коллизии	Ц (цепочки) и Дв (двойное хеширование)
n / m	30 000 / 49 999 (α ≈ 0,600)
Доп. задание	В6 — кластеризация при линейном пробировании

2. Описание хеш-функции и способов разрешения коллизий

2.1. Полиномиальное хеширование

$h(s) = (\Sigma\, s_i \cdot p^i) \bmod m$, $p = 31$. Зависит от всех символов → равномерное распределение. Плохой пример: h(s) = ord(s[0]) mod m — ~26 значений, катастрофические коллизии.

2.2. Метод цепочек

Элементы с одинаковым h(k) — в списке у ячейки. Допускается $\alpha > 1$. Средняя трудоёмкость поиска $\Theta(1+\alpha)$.

2.3. Двойное хеширование

$pos_i = (h_1(k) + i \cdot h_2(k)) \bmod m$. Обязательно $\alpha < 1$. Среднее число проб при неуспехе ≈ $1/(1-\alpha)$.

2.4. Линейное пробирование (для В6)

$pos_i = (h(k)+i) \bmod m$. Вызывает первичную кластеризацию — длинные занятые участки.

3. Схема структуры данных

Метод цепочек: массив из m списков; коллизии добавляются в список ячейки h(k). Двойное хеширование: элементы в самом массиве; при занятости — шаг $h_2(k)$.

4. Листинг программы (с пояснениями)

Полный код с комментариями к каждому действию — lab5-variant6/main.py.

4.1. Полиномиальный хеш

```
def poly_hash(key, m=M, p=31):
    h, pw = 0, 1
    for ch in key:
        h = (h + ord(ch) * pw) % m   # вклад символа
        pw = (pw * p) % m            # следующая степень
    return h
```

4.2. Вставка — метод цепочек

```
def insert(self, key, value):
    idx = self.hash_fn(key, self.m)
    chain = self.slots[idx]
    for i, (k, _) in enumerate(chain):
        self.probes += 1
        if k == key:
```

```
            chain[i] = (key, value); return   # обновление
    chain.append((key, value)); self.n += 1   # новый элемент
```

4.3. Вставка — двойное хеширование

```
def insert(self, key, value):
    pos, step = self._h1(key), self._h2(key)
    for _ in range(self.m):
        self.probes += 1
        k = self.keys[pos]
        if k is self.EMPTY or k is self.DELETED:
            self.keys[pos], self.values[pos] = key, value
            self.n += 1; return
        if k == key:
            self.values[pos] = value; return
        pos = (pos + step) % self.m
```

4.4. Кластеры (B6)

```
def cluster_lengths(self):
    occupied = [k is not self.EMPTY for k in self.keys]
    lengths, current = [], 0
    for flag in occupied + [False]:
        if flag: current += 1
        elif current: lengths.append(current); current = 0
    # кольцевой кластер
    if occupied[0] and occupied[-1] and len(lengths) &gt;= 2:
        lengths[0] += lengths[-1]; lengths.pop()
    return lengths
```

5. Характеристики вычислительной системы

Параметр	Значение
ОС	Linux
Язык	Python 3.12
Таймер	time.perf_counter()
Запросов на поиск	10 000 (успешные)

6. Результаты измерений

6.1. Основной эксперимент (n=30 000, m=49 999, α=0,600)

Реализация	Вставка, с	Поиск, с	Проб/запрос
Метод цепочек	0,044	0,013	1,30
Двойное хеширование	0,094	0,024	1,52
dict (эталон)	0,003	0,001	—

Цепочки: max длина = 6, пустых = 27 331 из 49 999, средняя = 0,600. Теория для успешного поиска при ОА: ½(1+1/(1−α)) ≈ 1,75; измерено 1,52.

6.2. Качественная vs плохая хеш-функция

Функция	Ненулевых ячеек	Макс. в ячейке
poly_hash	22 668	6
bad_hash (1-й символ)	26	1 215

6.3. Доп. B6 — кластеризация при линейном пробировании

α	Кластеров	Макс. длина	Средняя длина
0,3	9 071	12	1,65
0,5	9 942	37	2,51
0,7	7 537	91	4,64
0,9	2 932	440	15,35

С ростом α кластеры сливаются: число падает, длины резко растут — первичная кластеризация.

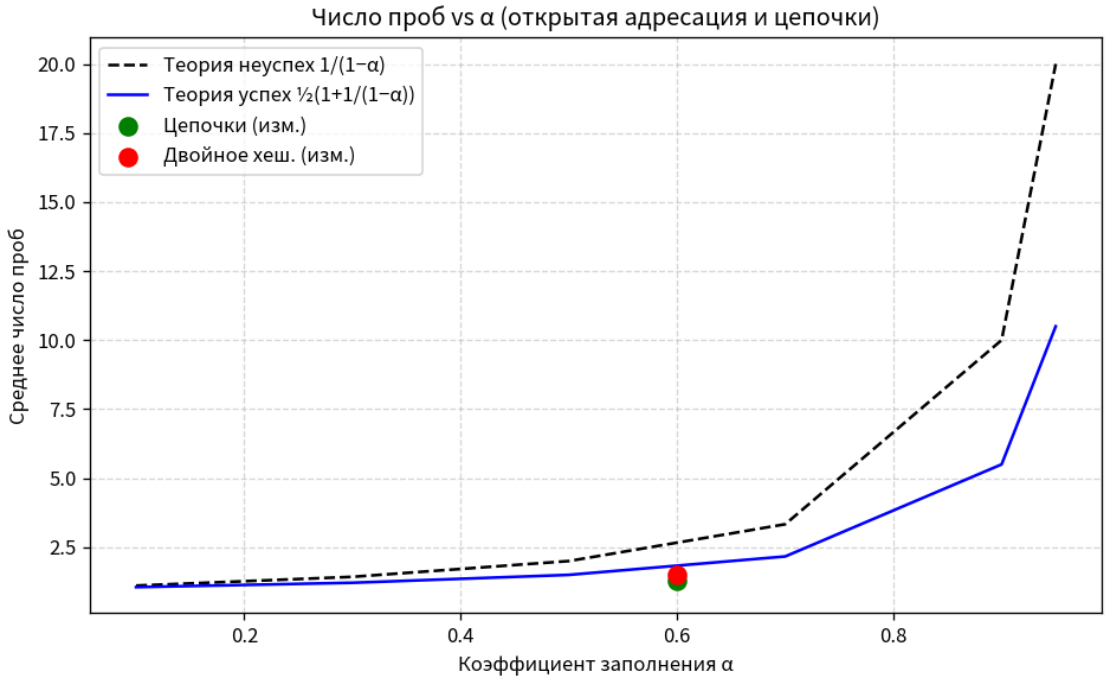


Рис. 1. Число проб vs α

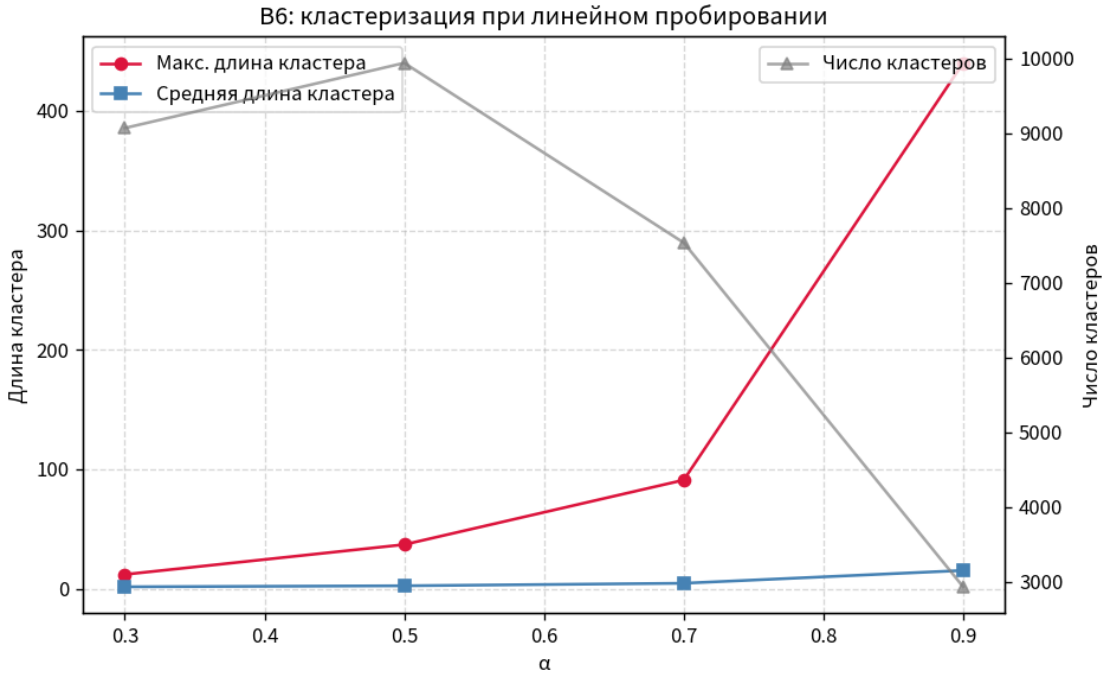


Рис. 2. Кластеризация при линейном пробировании (B6)

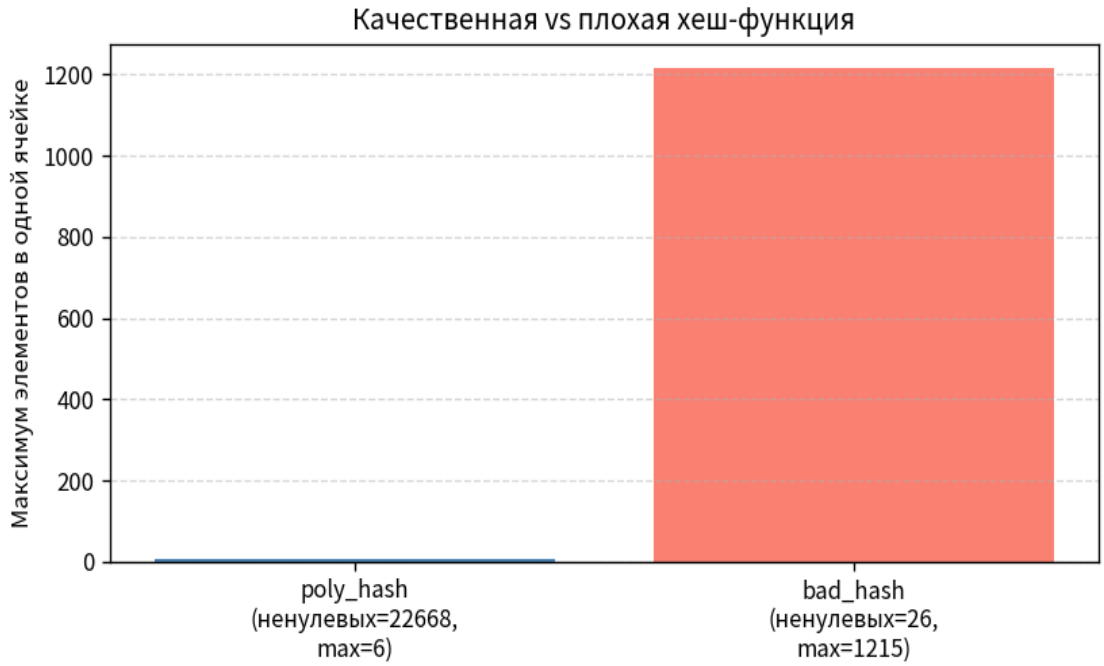


Рис. 3. Качественная vs плохая хеш-функция

7. Выводы

1. При α ≈ 0,6 обе реализации обеспечивают 1,3–1,5 пробы — согласуется с Θ(1+α).

2. Цепочки устойчивее к росту α; при открытой адресации число проб растёт как 1/(1−α).

3. Полиномиальный хеш равномерен (max 6); хеш по первому символу — max 1215.

4. В6: при линейном пробировании max длина кластера растёт с 12 до 440 при α от 0,3 до 0,9.

5. dict быстрее (C + оптимизации); своя таблица нужна для наблюдения внутренних метрик.

Конец отчёта.